MP3 Design Document

Overview

The following document outlines how I implemented MP3, including specifics on which files were modified, how certain data structures were implemented and details on specific algorithms. The frame manager is implemented entirely within the `ContFramePool` class while the page table manager within the `PageTable` class. The `PageTable` class implements a two-level x86-like page table manager that translates virtual addresses to physical frames on a two level hierarchy: page directory and page table. The first 4MB is shared and mapped one-to-one between physical and virtual addresses. while memory above 4MB is mapped when first used through page faults and interrupts. The kernel pool covers 2 - 4 MB while the process pool 4 - 32MB.

Data Structures

- Constants `PG_BIT`, `PAGE_TABLE_SPAN`, `PRESENT_BIT`, `WRITABLE_BIT`, `USER_BIT` - referenced by my implementation of the page table class. `PG_BIT` is CR0 bit 31 which turns paging on at the hardware level. `PAGE_TABLE_SPAN` is the 4MB of virtual memory one page table pages; P, R/W, and U/S are the low flag bits of an entry.

- `ContFramePool` - an implementation of a contiguous frame pool used in the implementation of `PageTable`. This implementation is identical to the implementation in my MP2 submission (2 bits per frame: Free, Used, Head-of-Sequence).

- Class members - `page_directory` is per instance member while `current_page_table`, `paging_enabled`, `kernel_mem_pool`, `process_mem_pool`, and `shared_size` are all static members.

- Page directory and page tables - each one is a single 4KB frame holding 1024 32-bit entries. Each entry holds a 4KB-aligned physical address in bits 31-12 and flags in bits 11-0. The frames for these tables are held in the kernel frame pool as the address space is one-to-one and which allows the kernel to reach them even after paging is on.

- Virtual address format - 10 / 10 / 12 bits split. The first 10 bits is an index into the page directory, the next 10 bits an index into the corresponding page table, and the last 12 bits an offset in the corresponding frame in physical memory.

init_pages

Initializes static variables in the `PageTable` class including the kernel and process memory pools and shared size member. Assertions are used to ensure that the passed addresses to `ContFramePool` instances are non-null and that the shared size argument is a multiple of the 4MB one page table maps (positive multiple of `PAGE_TABLE_SPAN`). A multiple of 4MB is

required here because the constructor maps the shared region one whole page table (4MB) at a time. Due to how this is computed (`shared_size / PAGE_TABLE_SPAN`), a size that isn't a multiple of 4MB would be truncated and thus leave the remainder unmapped. This method does not touch CR0 or CR3 and must run before the constructor so the static frame pools are ready to allocate frames.

Constructor (PageTable)

This method builds a new instance of the `PageTable` class with an address space in which the shared region is mapped and the rest is not present. Frames allocated in this method all come from the kernel memory pool as the kernel needs access to the page directory and page tables even after paging is turned on. One frame is used to store the page directory while multiple frames (depending on the size of the `shared_size` static member) are allocated for each page table (one frame per page table). All entries in the page table and the corresponding entry in the page directory are set as present and writable. The remaining entries in the page directory are only set as writable. CR0 and CR3 are left untouched, the responsibility for enabling paging and handling the page fault interrupt is on the `load` and `handle_fault` methods.

load

This method loads the address to the page directory into the CR3 register and updates the static member `current_page_table` to this `PageTable` instance. This should be called before `enable_paging` so that the CPU knows where to find the page directory.

enable_paging

This method updates the CR0 register to enable paging on the CPU. The `PG_BIT` constant is used to flip bit 31 and the `paging_enabled` static member is updated to indicate that paging is enabled on the CPU.

handle_fault

This method handles the page fault interrupt from the CPU. It first reads the `faulting_address` from the CR2 register and asserts that an access violation did not occur. In the case where the page fault was not due to an access violation, the method first retrieves the active page directory stored from initializing and loading a page table. Then it splits the faulting address into its corresponding page table components: first 10 bits for the directory index and middle 10 bits for the page table index. If a directory entry at the directory index is not present, a new directory entry is created. A frame from the kernel memory pool is allocated, the page directory entry is updated to present and writable, and a new page table is created with entries set to writable but not present. The pointer to the page table is then created using a bitwise AND

against `0xFFFFF000` to ensure that the flag bits do not offset the 4KB aligned address. A frame from the process memory pool is then allocated and inserted as an entry into the page table set as present and writable.

Files Modified and Justification

- `cont_frame_pool.H`: Same justification as MP2, added the private data members and helper method declaration listing in previous sections (`frame_state`, `base_frame_no`, `n_frames`, `n_free_frames`, `infor_frame_no`, `head`, `next`, `commit_frames`, `free_frames`) on top of the `FrameState` enum and `get_state` / `set_state` declarations.

- `cont_frame_pool.C`: Same justification as MP2, implemented every function stubbed out in the provided skeleton code (constructor, `get_frames`, `mark_inaccessible`, `release_frames`, `needed_info_frames`, `get_state`, `set_state`), plus two private helpers `commit_frames` and `free_frames` described above.

- `page_table.C`: Implements the methods outlined in `page_table.H` while also adding helper constants (constructor, `init_paging`, `load`, `enable_paging`, `handle_fault`)

- `kernel.C`: no code was added or written by me.